

Database Concepts and Skills for Big Data

Le Duy Dung (Andrew)

SQL – Part 7

Insert, Delete, Update

Insert Data

Inserting Data

As its name suggests, INSERT is used to insert (add) rows to a database table. Insert can be used in several ways:

- Inserting a single complete row
- Inserting a single partial row
- Inserting the results of a query

Inserting Complete Rows

The simplest way to insert data into a table is to use the basic INSERT syntax, which requires that you specify the table name and the values to be inserted into the new row.

<Example>

```
INSERT INTO CUSTOMER
VALUES(11, 'Andrew', 'Le', 'VinUniversity', 'Hanoi', 'HN', '99999',
'984780258', 'dung.ld@vinuni.edu.vn');
```

Inserting Data

<Example>

```
INSERT INTO CUSTOMER
VALUES(11, 'Andrew', 'Le', 'VinUniversity', 'Hanoi', 'HN', '99999',
'984780258', 'dung.ld@vinuni.edu.vn');
```

- The data to be stored in each table column is specified in the VALUES clause, and a value must be provided for every column.
- If a column has no value, the NULL value should be used (assuming the table allows no value to be specified for that column).
- The columns must be populated in the order in which they appear in the table definition.

Inserting Complete Rows

<Example>

```
INSERT INTO CUSTOMER
VALUES(11, 'Andrew', 'Le', 'VinUniversity', 'Hanoi', 'HN', '99999',
'984780258', 'dung.ld@vinuni.edu.vn');
```

- The above SQL statement is highly dependent on the order in which the columns are defined in the table.
- It also depends on information about that order being readily available. Even if it is available, there is no guarantee that the columns will be in the exact same order the next time the table is reconstructed.

Inserting Data

<Example>

```
INSERT INTO CUSTOMER (CustomerID, FirstName, LastName, Address, City,
State, ZIP, Phone, EmailAddress)
VALUES(12, 'Andrew', 'Le', 'VinUniversity', 'Hanoi', 'HN', '99999',
'984780258', 'dung.ld@vinuni.edu.vn');
```

- This example does the exact same thing as the previous INSERT statement, but this time the column names are explicitly stated in parentheses after the table name.
- When the row is inserted, the DBMS will match each item in the columns list with the appropriate value in the VALUES list.
- The first entry in VALUES corresponds to the first specified column name. The second value corresponds to the second column name, and so on.

Inserting Data

<Example>

```
INSERT INTO CUSTOMER (CustomerID, Address, City, State, ZIP, Phone, EmailAddress,
FirstName, LastName)

VALUES(13, 'VinUniversity', 'Hanoi', 'HN', '99999', '984780258', 'dung.ld@vinuni.edu.vn',
'Andrew', 'Le');
```

- Try this INSERT statement!!

Insert partial rows

- The recommended way to use INSERT is to explicitly specify table column names.
- Using this syntax, you can also omit columns. This means you provide values for only some columns, but not for others.

<Example>

```
INSERT INTO CUSTOMER (CustomerID, FirstName, LastName, Address, City, ZIP, Phone)
VALUES(14, 'Andrew', 'Le', 'VinUniversity', 'Hanoi', '99999', '984780258');
```

Update Data

Updating Data

To update (modify) data in a table, you use the UPDATE statement. UPDATE can be used in two ways:

- To update specific rows in a table
- To update all rows in a table

Updating Data

The basic format of an UPDATE statement is made up of three parts:

- The table to be updated
- The column names and their new values
- The filter condition that determines which rows should be updated

<Example>

```
UPDATE CUSTOMER
SET State = 'HN'
WHERE CustomerID = 14;
```

Updating Multiple Columns

The basic format of an UPDATE statement is made up of three parts:

- The table to be updated
- The column names and their new values
- The filter condition that determines which rows should be updated

<Example>

```
UPDATE CUSTOMER
SET City = 'Hanoi', EmailAddress='check@gmail.com'
WHERE CustomerID = 14;
```

Using Subqueries in an UPDATE Statement

Assume that Vendor 1 has bought Vendor 2. Your store needs to update the ITEM table, change the VendorID with value of 2 to value 1.

<Example>

```
UPDATE ITEM
SET VendorID=1
WHERE ItemID IN (SELECT ItemID FROM ITEM WHERE VendorID=2);
```

Delete Data

Deleting Data

To delete (remove) data from a table, you use the DELETE statement. DELETE can be used in two ways:

- To delete specific rows from a table
- To delete all rows from a table

Deleting Data

Assume that Vendor 1 has bought Vendor 2. Your store needs to update the ITEM table, change the VendorID with value of 2 to value 1.

<Example>

```
DELETE FROM VENDOR
WHERE VendorID=2;
```

Guidelines for Updating and Deleting

- Never execute an UPDATE or a DELETE without a WHERE clause unless you really do intend to update and delete every row.
- Make sure every table has a primary key (refer to Lesson 12 if you have forgotten what this is), and use it as the WHERE clause whenever possible. (You may specify individual primary keys, multiple values, or value ranges.)
- Before you use a WHERE clause with an UPDATE or a DELETE, first test it with a SELECT to make sure it is filtering the right records; it is far too easy to write incorrect WHERE clauses.
- Use database-enforced referential integrity (refer to Lesson 12 for this one too) so that the DBMS will not allow the deletion of rows that have data in other tables related to them.
- Some DBMSs allow database administrators to impose restrictions that prevent the execution of UPDATE or DELETE without a WHERE clause. If your DBMS supports this feature, consider using it.



QnA!